JADE Framework

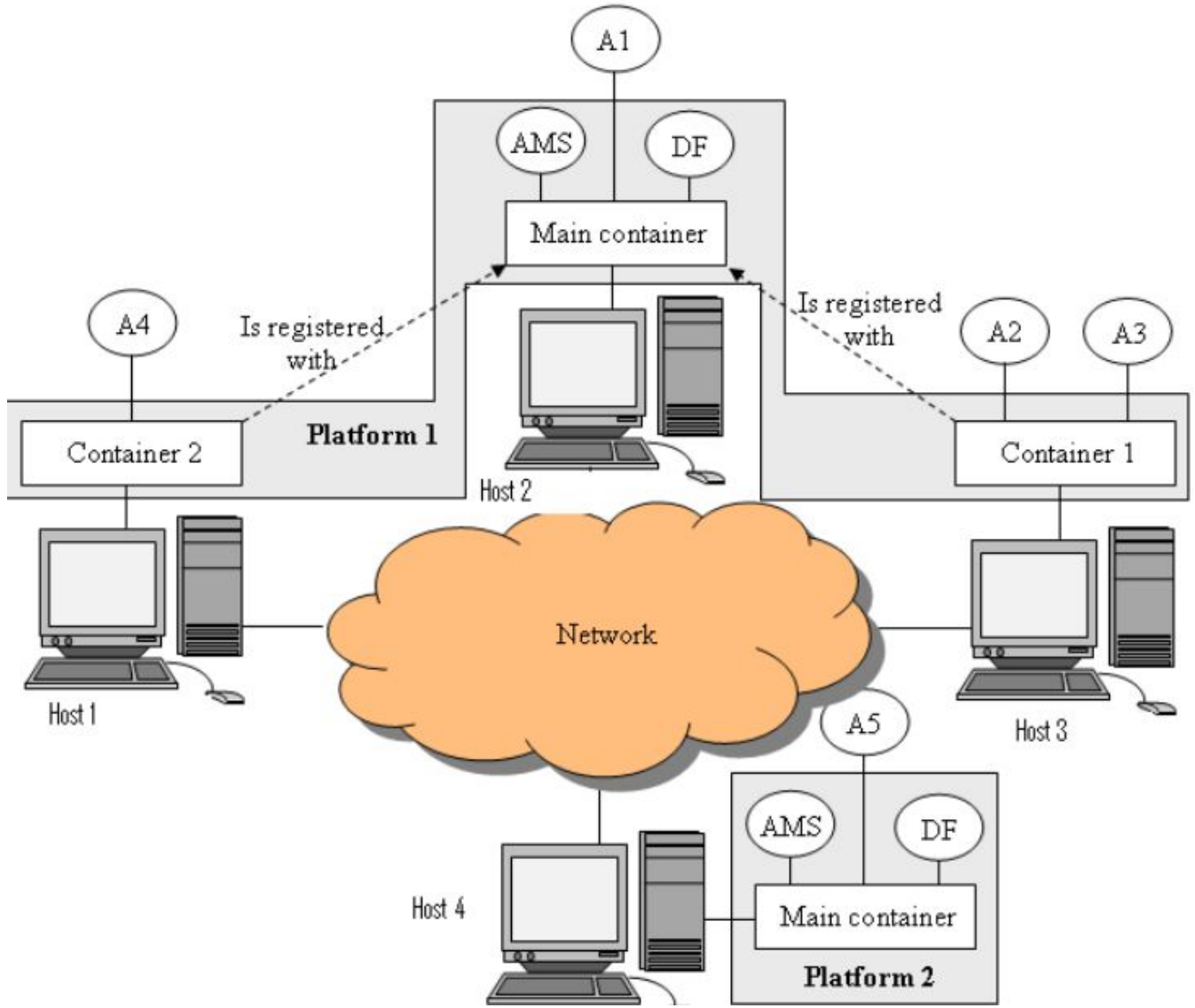
Dr. Kaneeka Vidanage.

Dept of CS, FOC, KDU

Introduction

- Simplifies the implementation of Multi Agent Systems (MAS) adhering to the FIPA [**F**oundation for **I**ntelligent **P**hysical **A**gents] standards.
- Requires JRE, greater than Version 05.
- JADE handles ACL [**A**gent **C**ommunication **L**anguage]communications between agents efficiently.
- JADE framework comprises of multiple important components.
  - **Platform** Allows the loaded agents to live and communicate.
  - **Main Container** Inside the platform and the 1st container to start. Contains more special components as : -
    - **AMS** Agent Management System Supervises and maintains agents states and IDs (AIDs).
    - **DF** Directory Facilitator Acts as a yellow page service by registering and de-registering agent services
  - **Containers** Inside the platform and links with the Main Container to commence the operations.

JADE Architecture



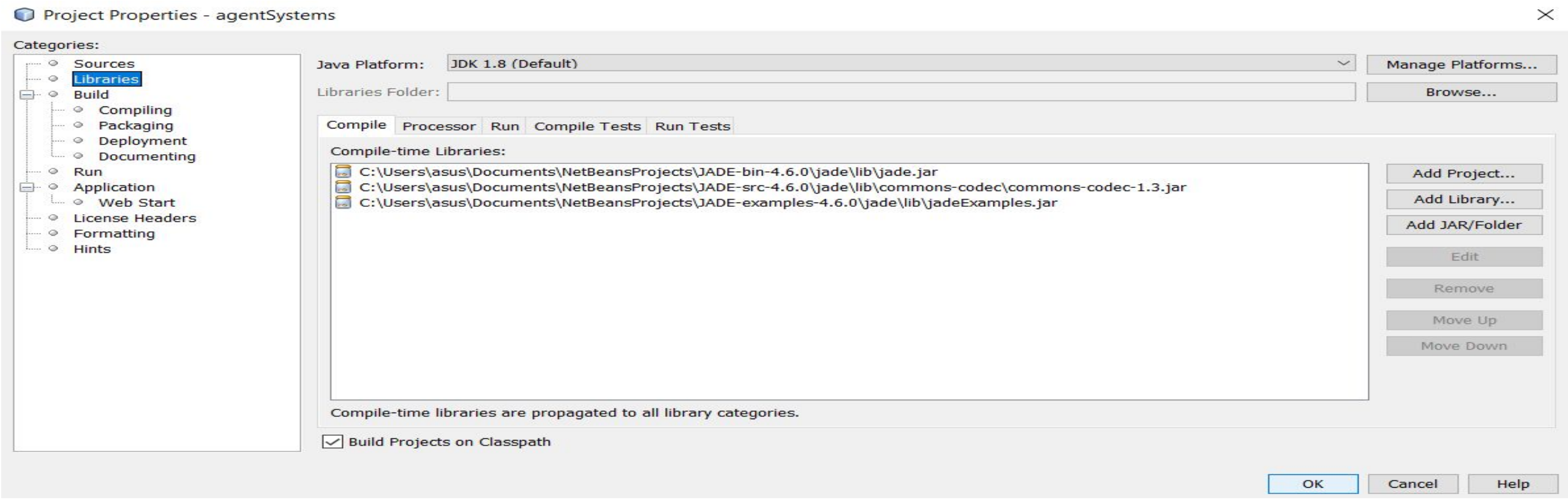
Agent Communication

- Communication occurs via ACL messages.
- ACL message contains : -
  - **Sender** details.
  - **Receiver** details.
  - **Communicative Act** / **Performative**.
    - INFORM Intends receiver to be aware about some info / state / process.
    - REQUEST Expects receiver agent to perform some action
    - QUERY_IF Checks / Asks for specific information from an agent
    - CFP Call for Proposals / Buyer agents requests for offers from Seller agents.
    - PROPOSE Contains seller`s offers to buyers.
    - ACCEPT_PROPOSAL Specific seller offer is accepted by specific buyer
  - **Content** Information included in the msg.
  - **Language** Syntax used to express the content. Both sender & receiver should use a common language for the comprehension.
  - **Ontology** Schematics and vocabulary linking the aspects in the domain to make the messages validated and more meaningful.

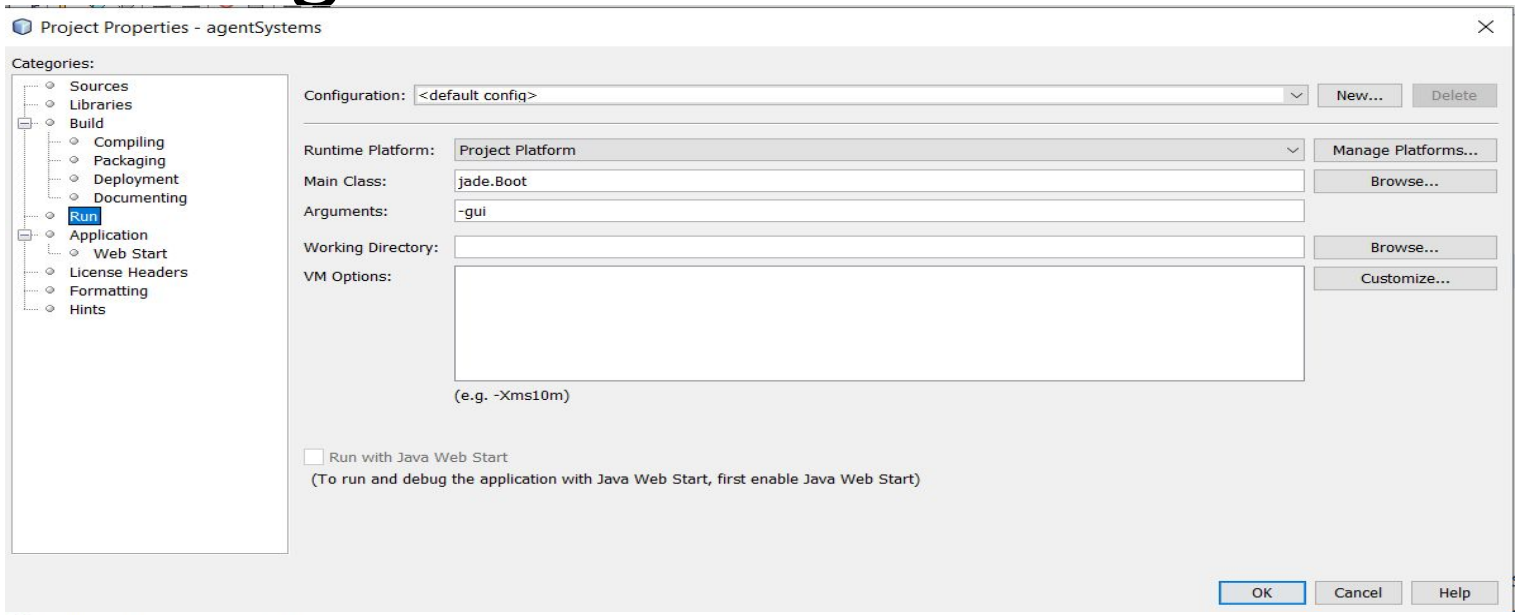
Installing and Starting JADE

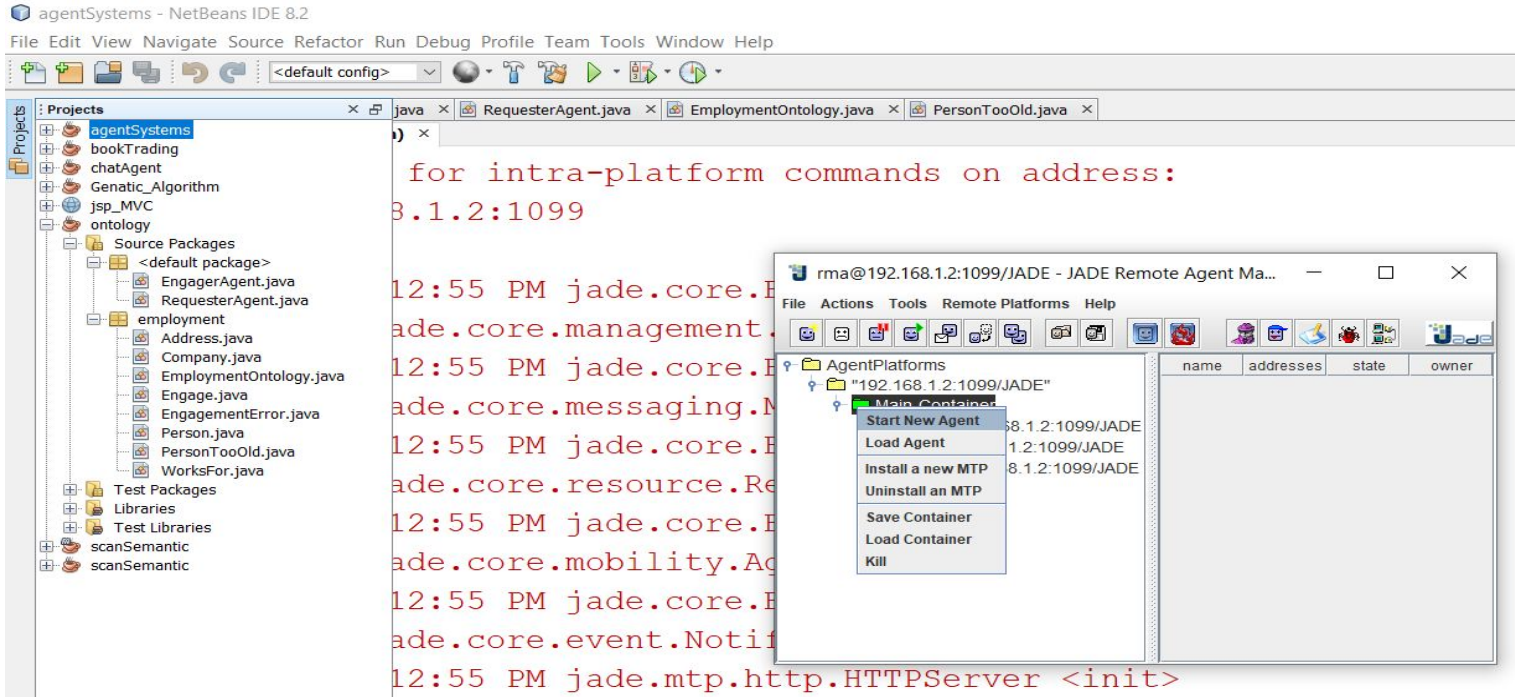
- Download via https://jade.tilab.com/
- Unzip and Extract.
- Add the relevant jars to the NetBeans / Eclipse project created.

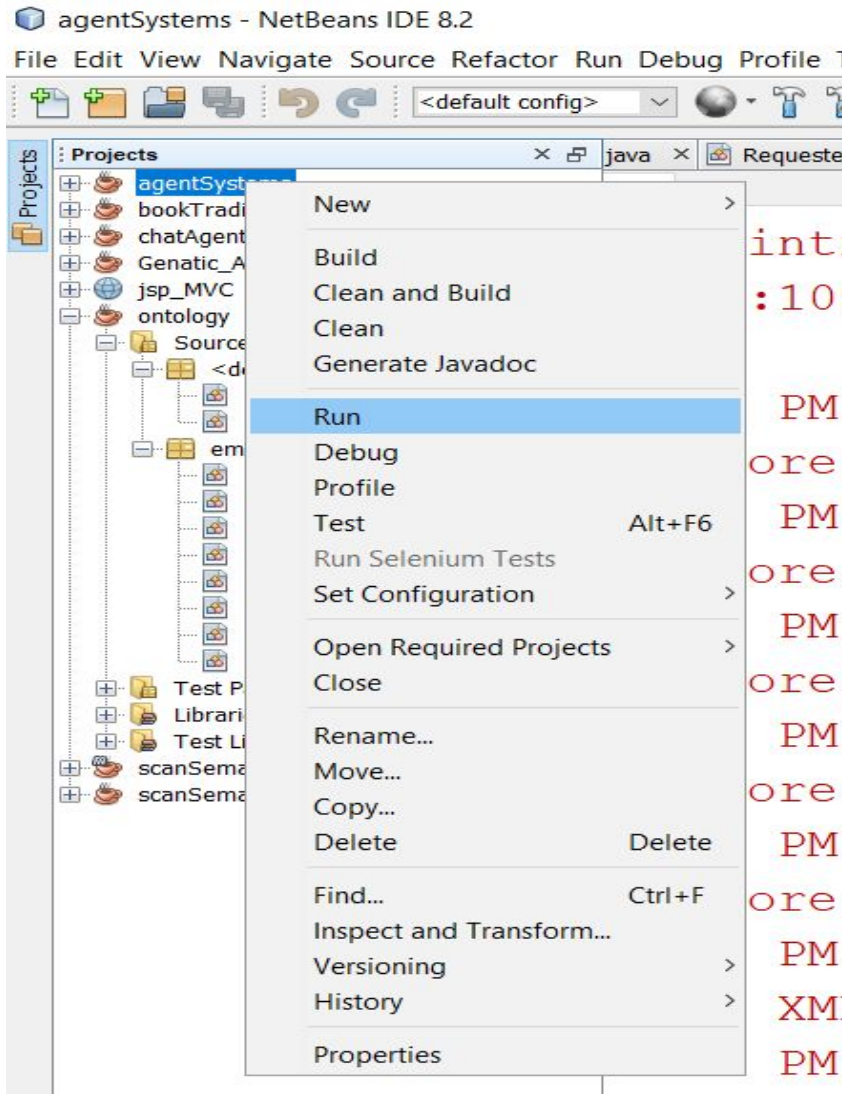




Starting JADE & JADE GUI Console



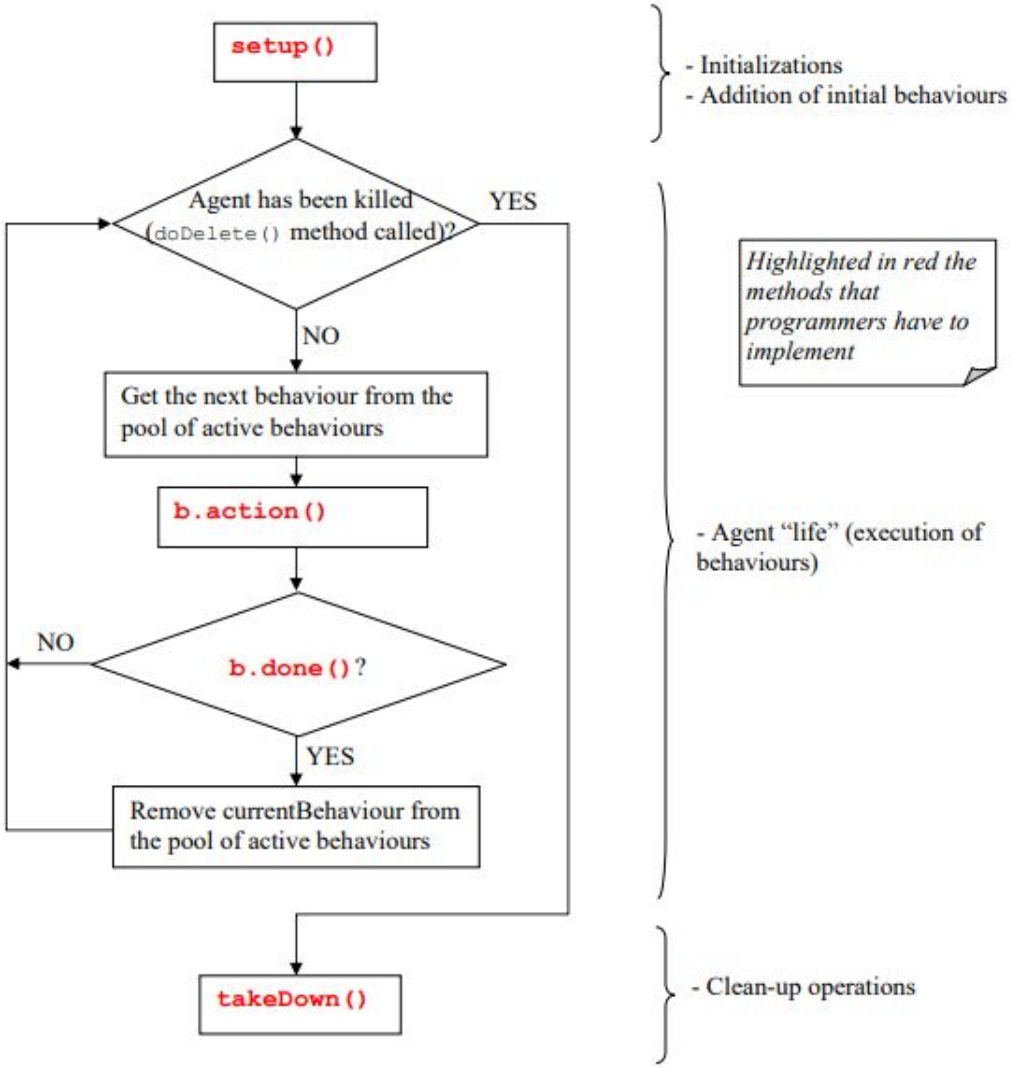




Agent Behaviors

- Naturally, tasks associated with an agent are carried out as behaviors.
- There are various types of behaviors, used for various purposes.
- Custom agents' behaviors are extended from **jade.core.behaviours.Behaviour.**
- In order to make an agent execute the **task implemented by a behavior** object
  - it is sufficient to add the behavior to the agent via the **addBehaviour()** method of the **Agent** class
- Each class extending Behavior must implement the **action()** method, that defines the operations to be performed by the agent.
- After, the **done()** method (returns a Boolean value), specifies whether the behavior has completed or not.

Agent Execution Sequence



Behavior Types

- **One-shot** complete immediately the action() method.
  - It`s important to state, that it will be executed **only once**.
  - Once the **one-time** execution completes, automatically the done method will return true.

```
public class MyOneShotBehaviour extends OneShotBehaviour {
  public void action() {
    // perform operation X
  }
}
```

Operation X is performed only once.

- **Cyclic**

```
public class MyCyclicBehaviour extends CyclicBehaviour {
  public void action() {
    // perform operation Y
  }
}
```

Operation Y is performed repetitively forever (until the agent carrying out the above behaviour terminates).

  - Done method will return false, upon completion of action method`s execution.

Behavior Types

- **Generic Behavior** Stops once a defined condition is met.

```
public class MyThreeStepBehaviour extends Behaviour {
  private int step = 0;
  public void action() {
    switch (step) {
    case 0:
      // perform operation X
      step++;
      break;
    case 1:
      // perform operation Y
      step++;
      break;
    case 2:
      // perform operation Z
      step++;
      break;
    }
  }

  public boolean done() {
    return step == 3;
  }
}
```

Operations X, Y and Z are performed one after the other and then the behaviour completes.

Behavior Types

- **Waker Behavior** □ Executes, when time elapse is reached.

```
public class MyAgent extends Agent {
  protected void setup() {
    System.out.println("Adding waker behaviour");
    addBehaviour(new WakerBehaviour(this, 10000) {
      protected void handleElapsedTimeout() {
        // perform operation X
      }
    } );
  }
}
```

Operation X is performed 10 seconds after the "Adding waker behaviour" printout appears.

Behavior Types

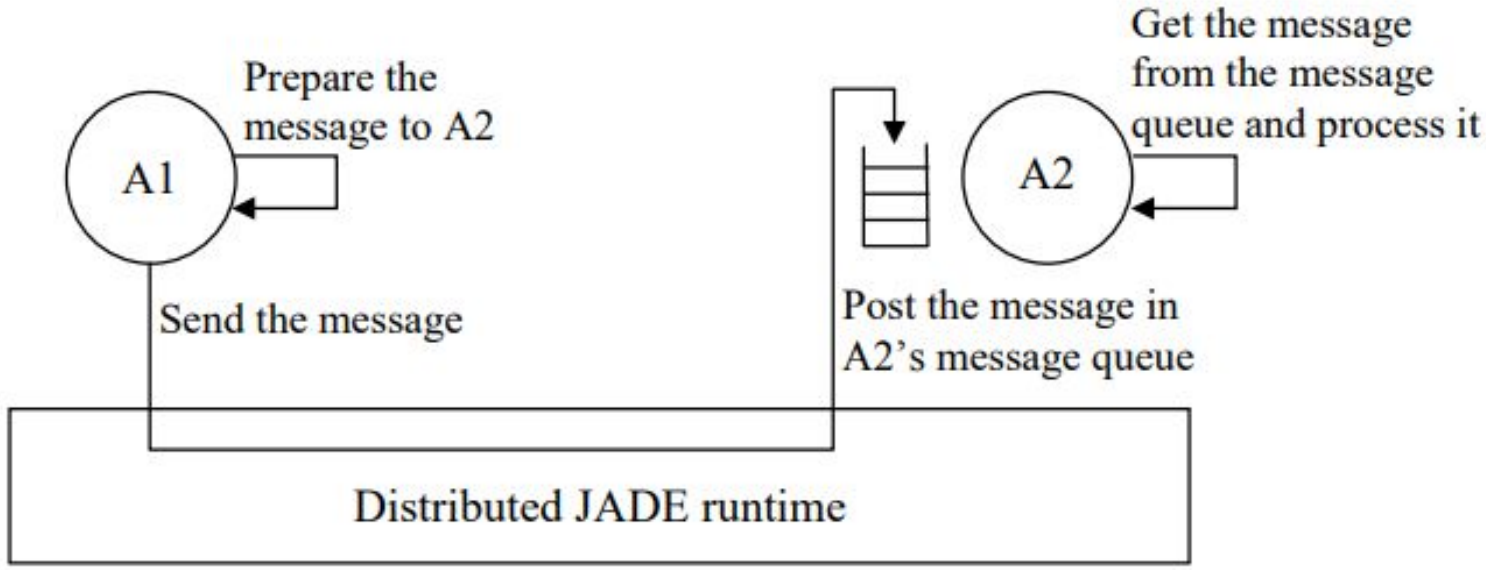
- **Ticker Behavior**

```
public class MyAgent extends Agent {
  protected void setup() {
    addBehaviour(new TickerBehaviour(this, 10000) {
      protected void onTick() {
        // perform operation Y
      }
    } );
  }
}
```

Operation Y is performed periodically every 10 seconds.

ACL Messages

One of the most important features that JADE agents provide is the ability to communicate. The communication paradigm adopted is the **asynchronous message passing**. Each agent has a sort of mailbox (the agent message queue) where the JADE runtime posts messages sent by other agents. Whenever a message is posted in the message queue the receiving agent is notified. If and when the agent actually picks up the message from the message queue to process it is completely up to the programmer however.



Agent Communication

- Communication occurs via ACL messages.
- ACL message contains : -
  - **Sender** details.
  - **Receiver** details.
  - **Communicative Act** / **Performative**.
    - INFORM Intends receiver to be aware about some info / state / process.
    - REQUEST Expects receiver agent to perform some action
    - QUERY_IF Checks / Asks for specific information from an agent
  - **Content** Information included in the msg.
  - **Language** Syntax used to express the content. Both sender & receiver should use a common language for the comprehension.
  - **Ontology** Schematics and vocabulary linking the aspects in the domain to make the messages validated and more meaningful.

Sending msgs via ACL

Sending a message to another agent is as simple as filling the fields of an `ACLMessage` object and then call the `send()` method of the `Agent` class. The code below informs an agent whose nickname is *Peter* that *today it's raining*.

```
ACLMessage msg = new ACLMessage(ACLMessage.INFORM);
msg.addReceiver(new AID("Peter", AID.ISLOCALNAME));
msg.setLanguage("English");
msg.setOntology("Weather-forecast-ontology");
msg.setContent("Today it's raining");
send(msg);
```

```
// Message carrying a request for offer
ACLMessage cfp = new ACLMessage(ACLMessage.CFP);
for (int i = 0; i < sellerAgents.lenght; ++i) {
  cfp.addReceiver(sellerAgents[i]);
}
cfp.setContent(targetBookTitle);
myAgent.send(cfp);
```

```
ACLMessage msg = receive();
if (msg != null) {
  // Process the message
}
```

Logic for serving incoming requests by Buyers

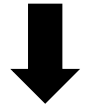
- Seller agent must continuously, serve the messages from buyer agents.
- Hence, it`s continuous, such behavior must be cyclic behaviors.
- Execution of their action() method, must check if a message has been received and process it.

```
/**
   Inner class OfferRequestsServer.
   This is the behaviour used by Book-seller agents to serve incoming requests
   for offer from buyer agents.
   If the requested book is in the local catalogue the seller agent replies
   with a PROPOSE message specifying the price. Otherwise a REFUSE message is
   sent back.
 */
private class OfferRequestsServer extends CyclicBehaviour {
  public void action() {
    ACLMessage msg = myAgent.receive();
    if (msg != null) {
      // Message received. Process it
      String title = msg.getContent();
      ACLMessage reply = msg.createReply();

      Integer price = (Integer) catalogue.get(title);
      if (price != null) {
        // The requested book is available for sale. Reply with the price
        reply.setPerformative(ACLMessage.PROPOSE);
        reply.setContent(String.valueOf(price.intValue()));
      }
      else {
        // The requested book is NOT available for sale.
        reply.setPerformative(ACLMessage.REFUSE);
        reply.setContent("not-available");
      }
      myAgent.send(reply);
    }
  }
}  // End of inner class OfferRequestsServer
```

To avoid the execution of the action() method of the OfferRequestsServer continuous (i.e., as of the cyclic behavior) we can use the **block()** method of the Behavior class.

Unless CPU power is wasted.



```
public void action() {
  ACLMessage msg = myAgent.receive();
  if (msg != null) {
    // Message received. Process it
    ...
  }
  else {
    block();
  }
}
```

Message Template

- Inside a container, hence, multiple agents' function, lots of information travel.
- We need to have a conditionally filtering mechanism to ensure specific agents only respond to specific msgs only, via ignoring all others.
- This requirement can be achieved via Message Templates.
- Here, the action will process only the CFP, message categories.

```
public void action() {
  MessageTemplate mt = MessageTemplate.MatchPerformative(ACLMessage.CFP);
  ACLMessage msg = myAgent.receive(mt);
  if (msg != null) {
    // CFP Message received. Process it
    ...
  }
  else {
    block();
  }
}
```

Complex Conversation Handling.

- A conversation is a sequence of messages exchanged by two or more agents.
  1. Buyer agents **send** the CFP to all sellers.
  2. Buyer agents **receive** all proposals/refusals from **seller** agents
  3. Buyer agent send the purchase order to the seller that provided the best offer
  4. Buyer agent receive the purchase order reply from the best offer seller.

1. Buyer agents send the CFP to all sellers.

```
/**
   Inner class RequestPerformer.
   This is the behaviour used by Book-buyer agents to request seller
   agents the target book.
 */
private class RequestPerformer extends Behaviour {
  private AID bestSeller; // The agent who provides the best offer
  private int bestPrice;  // The best offered price
  private int repliesCnt = 0; // The counter of replies from seller agents
  private MessageTemplate mt; // The template to receive replies
  private int step = 0;

  public void action() {
    switch (step) {
    case 0:
      // Send the cfp to all sellers
      ACLMessage cfp = new ACLMessage(ACLMessage.CFP);
      for (int i = 0; i < sellerAgents.length; ++i) {
        cfp.addReceiver(sellerAgents[i]);
      }
      cfp.setContent(targetBookTitle);
      cfp.setConversationId("book-trade");
      cfp.setReplyWith("cfp"+System.currentTimeMillis()); // Unique value
      myAgent.send(cfp);
      // Prepare the template to get proposals
      mt = MessageTemplate.and(MessageTemplate.MatchConversationId("book-trade"),
                               MessageTemplate.MatchInReplyTo(cfp.getReplyWith()));
      step = 1;
      break;
```

2. Buyer agents receive all proposals/refusals from seller agents

```
case 1:
  // Receive all proposals/refusals from seller agents
  ACLMessage reply = myAgent.receive(mt);
  if (reply != null) {
    // Reply received
    if (reply.getPerformative() == ACLMessage.PROPOSE) {
      // This is an offer
      int price = Integer.parseInt(reply.getContent());
      if (bestSeller == null || price < bestPrice) {
        // This is the best offer at present
        bestPrice = price;
        bestSeller = reply.getSender();
      }
    }
    repliesCnt++;
    if (repliesCnt >= sellerAgents.length) {
      // We received all replies
      step = 2;
    }
  }
  else {



    block();
  }
  break;
```

3. Buyer agent send the purchase order to the seller that provided the best offer

```
case 2:
  // Send the purchase order to the seller that provided the best offer
  ACLMessage order = new ACLMessage(ACLMessage.ACCEPT_PROPOSAL);
  order.addReceiver(bestSeller);
  order.setContent(targetBookTitle);
  order.setConversationId("book-trade");
  order.setReplyWith("order"+System.currentTimeMillis());
  myAgent.send(order);
  // Prepare the template to get the purchase order reply
  mt = MessageTemplate.and(MessageTemplate.MatchConversationId("book-trade"),
                           MessageTemplate.MatchInReplyTo(order.getReplyWith()));
  step = 3;
  break;
```

4. Buyer agent receive the purchase order reply from the best offer seller.

```
      case 3:
        // Receive the purchase order reply
        reply = myAgent.receive(mt);
        if (reply != null) {
          // Purchase order reply received
          if (reply.getPerformative() == ACLMessage.INFORM) {
            // Purchase successful. We can terminate
            System.out.println(targetBookTitle+" successfully purchased.");
            System.out.println("Price = "+bestPrice);
            myAgent.doDelete();
          }
          step = 4;
        }
        else {
          block();
        }
        break;
      }
    }

    public boolean done() {
      return ((step == 2 && bestSeller == null) || step == 4);
    }
  }  // End of inner class RequestPerformer
```